Exercices supplémentaires

⋆ ⋆ ⋆

Exercice 1.

Inverser, quand cela est possible, les applications suivantes :

$$\begin{array}{rcl} c_1 : \mathbb{Z}/11\mathbb{Z} & \to & \mathbb{Z}/11\mathbb{Z} \\ x & \to & c_1(x) = 2x \end{array} \qquad \begin{array}{rcl} c_2 : \mathbb{Z}/11\mathbb{Z} & \to & \mathbb{Z}/11\mathbb{Z} \\ x & \to & c_2(x) = 4x + 8 \end{array}$$

$$\begin{array}{rcl} c_3 : \mathbb{Z}/15\mathbb{Z} & \to & \mathbb{Z}/15\mathbb{Z} \\ x & \to & c_3(x) = 5x + 7 \end{array} \qquad \begin{array}{rcl} c_4 : \mathbb{Z}/15\mathbb{Z} & \to & \mathbb{Z}/15\mathbb{Z} \\ x & \to & c_4(x) = 7x + 5 \end{array}$$

$$\begin{array}{rcl} c_5 : \mathbb{Z}/21\mathbb{Z} & \to & \mathbb{Z}/21\mathbb{Z} \\ x & \to & c_5(x) = 14x + 9 \end{array} \qquad \begin{array}{rcl} c_6 : \mathbb{Z}/21\mathbb{Z} & \to & \mathbb{Z}/21\mathbb{Z} \\ x & \to & c_6(x) = 4x + 6 \end{array}$$

Exercice 2.

1. De $\mathbb{Z}/30\mathbb{Z}$ dans $\mathbb{Z}/30\mathbb{Z}$:
- combien peut-on construire de fonctions bijectives ?
- combien peut-on construire de fonctions affines bijectives ?

2. Sur l'ensemble des caractères ASCII standard (128 caractères), on met en place un chiffrement symétrique en utilisant un chiffrement affine. Combien y-a-t-il de clés possibles ?

Exercice 3.

On travaille avec un alphabet réduit aux 26 lettres majuscules de A à Z.

1. Ecrire une fonction Python qui prend en entrée une chaîne de caractères ainsi qu'une clé k et qui renvoie le chiffré avec César.

2. On donne le texte suivant chiffré avec César : DAPQLMHQKDAZ
Par **force brute**, afficher tous les déchiffrements possibles.

3. Ecrire un script permettant de réaliser une attaque par analyse fréquentielle sur un texte chiffré avec César. Il faut donc déterminer la lettre qui apparaît le plus souvent dans le chiffré. Cette lettre est alors très probablement le chiffré du E. Et on peut alors proposer la clé la plus probable.

Exercice 4.

Ecrire, en Python, un script permettant de décrypter un texte chiffré avec Vigenère, en utilisant une analyse fréquentielle. On suppose que la longueur de la clé est connue (on la note k).

Vous devez donc :
→ Découper le texte chiffré en k sous-textes
→ Pour chaque sous-texte, mener une analyse fréquentielle et déduire ainsi les k caractères de la clé.

Exercice 5.

Ecrire, en Python, un script permettant à partir d'un texte (suffisamment long) chiffré avec Vigenère, de proposer une longueur probable pour la clé de chiffrement.

On mettra en oeuvre un test de Kasiski.

Exercice 6.

Mettre en évidence qu'un chiffrement de Vigenère permet une diffusion des propriétés statistiques des textes en clair, quand ils sont chiffrés.

Pour cela, on pourra :
• S'appuyer sur un texte en clair (suffisamment long).
• Etablir une analyse fréquentielle des caractères de ce texte ; vous devez ici retrouver, plus ou moins, les fréquences d'apparition de chaque lettre, propres à la langue française.
• Chiffrer le texte avec Vigenère en utilisant une clé donnée.
• Etablir de nouveau une analyse fréquentielle des caractères du texte chiffré.
→ Vous devez observer une uniformisation de la distribution des caractères suite au chiffrement.
• Jouer sur la longueur de la clé de chiffrement et observer que plus la longueur de la clé est grande, plus (dans le chiffré) la distribution des caractères tend vers une distribution uniforme.

Exercice 7.

1. Calculer le pgcd de 457 et 26. En déduire les coefficients de Bézout de 457 et 26.
(Utiliser Euclide et Euclide étendu.)

2. Calculer les coefficients de Bézout de 1233 et 5671. En déduire l'inverse de 1233 modulo 5671.
(Utiliser Euclide, Euclide étendu puis en déduire l'inverse.)

3. Calculer l'inverse de 331 modulo 3450.

Exercice 8.

L'idée est ici de calculer un inverse modulaire en utilisant le petit théorème de Fermat.

1. Déterminer les inverses suivants (directement ou avec Euclide puis Euclide étendu) :
- Inverse de 3 dans $\mathbb{Z}/7\mathbb{Z}$
- Inverse de 4 dans $\mathbb{Z}/11\mathbb{Z}$
- Inverse de 7 dans $\mathbb{Z}/26\mathbb{Z}$

2. On rappelle le petit théorème de Fermat :

Si p est un nombre premier, alors pour tout entier a tel que $1 \leq a < p$ on a $a^{p-1} \equiv 1\ [p]$

Puisque $a^{p-1} \equiv 1\ [p]$, on peut écrire $a \times a^{p-2} \equiv 1\ [p]$ et ainsi, l'inverse de a dans $\mathbb{Z}/p\mathbb{Z}$ est a^{p-2}
Cela permet de trouver un inverse dans $\mathbb{Z}/p\mathbb{Z}$, à condition que p soit premier.
La méthode est rapide car il est facile de calculer une exponentiation modulaire (le a^{p-2}).

Appliquer cette méthode pour (re)trouver l'inverse de 3 dans $\mathbb{Z}/7\mathbb{Z}$, puis celui de 4 dans $\mathbb{Z}/11\mathbb{Z}$.
Avec cette méthode, la recherche de l'inverse de 7 dans $\mathbb{Z}/26\mathbb{Z}$ n'est pas possible puisque 26 n'est pas premier.

En utilisant cette méthode, trouver l'inverse de 8 dans $\mathbb{Z}/13\mathbb{Z}$.

Exercice 9.

Cet exercice présente le test de primalité de Fermat. C'est un test probabiliste.
On rappelle le petit théorème de Fermat :

Si p est un nombre premier premier, alors pour tout entier a tel que $1 \leq a < p$ on a $a^{p-1} \equiv 1\ [p]$

Ainsi s'il existe un a, avec $1 \leq a < p$, vérifiant $a^{p-1} \neq 1[p]$, alors p n'est pas premier.
$\rightarrow$ une telle valeur de a est un "témoin" qui affirme la non primalité de p

On peut alors construire un test de primalité probabiliste en procédant comme suit :
1. Choisir un nombre aléatoire a avec $1 < a < p$
2. Calculer $a^{p-1}[p]$
3. Si le résultat n'est pas égal à 1, alors p n'est pas premier
   $\Rightarrow$ le test s'arrête, on est certain que p n'est pas premier
4. Si le résultat est égal à 1, alors la probabilité que p soit premier augmente ...
   $\Rightarrow$ si on le souhaite, retour à l'étape **1.** pour renforcer (éventuellement) la probabilité que p soit effectivement premier

Il faut bien comprendre que ce test est probabiliste.
Il ne pourra pas certifier que p est effectivement premier avec une probabilité égale à 1.
L'intérêt d'une telle méthode est qu'elle est plus rapide que les tests de primalité naïfs (basés sur la recherche d'éventuels diviseurs).
Le test de Miller-Rabin (plus performant) est aussi un test de primalité probabiliste.

Travail à faire :
Ecrire une fonction en Python qui implémente le test de primalité de Fermat.
Votre fonction prendra p (à tester) en entrée et testera toutes les valeurs de a de 2 à 50.
Si un témoin de non primalité est trouvé, la fonction indique en sortie que p n'est pas premier ; sinon, elle indique que p est *probablement* premier.

Exercice 10.

Ecrire un script Python permettant de chiffrer et déchiffrer un texte en utilisant un chiffrement de Hill.
On limitera la taille de l'alphabet à 26 caractères et on utilisera un chiffrement par paquet de 3 lettres avec la matrice de chiffrement $M = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 3 & 1 \\ 1 & 1 & 2 \end{pmatrix}$

Dans un premier temps, on pourra vérifier que dans $\mathbb{Z}/26\mathbb{Z}$ la matrice inverse de M est donnée par $M^{-1} = \begin{pmatrix} 23 & 18 & 4 \\ 15 & 4 & 11 \\ 7 & 15 & 19 \end{pmatrix}$

Exercice 11.

Se renseigner sur les algorithmes de chiffrement symétrique effectivement utilisés :

- DES (Data Encryption Standard), choisi comme standard dans les années 1970
- AES (Advanced Encryption Standard), qui remplace aujourd'hui le DES.

Exercice 12.

Ecrire une fonction python permettant de calculer une exponentiation modulaire.
Pour ce faire, on demande d'utiliser l'algorithme adéquat.

Exercice 13.

Les calculs pourront être conduits avec Python.

On donne le cryptosystème RSA suivant :

- clé publique $(n, e) = (100354644349273, 5)$
- clé privée $(n, d) = (100354644349273, 60212774548685)$

1. Vérifier que le chiffrement de l'entier 817289163 avec la clé publique donne l'entier 25877066682002
2. Vérifier que le déchiffrement de l'entier 25877066682002 avec la clé privée (re)donne bien ce qui est attendu.
3. Quels sont les deux entiers premiers qui ont été utilisés pour construire ce cryptosystème ?

Exercice 14.

1. Vérifier que les seuls générateurs du groupe $((\mathbb{Z}/13\mathbb{Z})^*, \times)$ sont 2, 6, 7 et 11.

2. Déterminer tous les générateurs du groupe $((\mathbb{Z}/23\mathbb{Z})^*, \times)$

Exercice 15.

Soit p un entier premier et $((\mathbb{Z}/p\mathbb{Z})^*, \times)$ le groupe multiplicatif modulo p.
On se donne g un entier entre 2 et $p-1$ et on veut savoir si g est un générateur du groupe $((\mathbb{Z}/p\mathbb{Z})^*, \times)$

Pour ce faire, on peut mettre en place le test suivant :

- Déterminer tous les diviseurs premiers q_1, q_2, ... q_k de $p-1$
  Par exemple, dans $((\mathbb{Z}/13\mathbb{Z})^*, \times)$, $p = 13$ donc $p-1 = 12$ et on a $q_1 = 2$ et $q_2 = 3$.
- Pour chaque diviseur premier q_i, calculer $a^{\frac{p-1}{q_i}}$ modulo p.
  $\Longrightarrow$ si aucun de ces résultats n'est égal à 1, alors g est un générateur du groupe $((\mathbb{Z}/p\mathbb{Z})^*, \times)$
  $\Longrightarrow$ sinon (au moins l'un des calculs donne 1), alors g n'est pas un générateur.

1. En utilisant ce test, vérifier que 7 est un générateur du groupe $((\mathbb{Z}/p\mathbb{Z})^*, \times)$
2. En utilisant ce test, vérifier que 5 n'est pas un générateur du groupe $((\mathbb{Z}/p\mathbb{Z})^*, \times)$

3. Ecrire un script Python permettant de tester si un entier g donné est un générateur du groupe multiplicatif $((\mathbb{Z}/p\mathbb{Z})^*, \times)$ où p est un entier premier donné.

Exercice 16.

Se renseigner sur la cryptographie asymétrique utilisant des courbes elliptiques.